Progress Report

- Increment 3 -

Group #5

1) Team Members

Fu, Yulissa (FSU ID: yaf22b) (GitHub: yulissafu@gmail.com)

Hsieh, Alexander (FSU ID: ach22h) (GitHub: alex-hsieh)

Metzger, Brooklyn (FSU ID: bem23b) (GitHub: brooklynmetzger@yahoo.com)

Reiter, Olivia (FSU ID: or22a) (GitHub: or22a@fsu.edu)

Stavreski, Aleksandar (FSU ID: as21em) (GitHub: as21emAS)

Su, Julissa (FSU ID: js22cu) (GitHub: julissa.su01@gmail.com)

2) Project Title and Description

RACETRACK is a full-stack web application that uses machine learning to predict Formula 1 race outcomes. The system processes historical race data spanning 17 seasons (2010–2026) — over 3,500 race records — and surfaces predictions through an interactive, data-driven interface. The application is composed of a React/TypeScript frontend, a FastAPI/Python backend, a PostgreSQL database, and a scikit-learn ML pipeline, and is fully deployed to production with the frontend and backend both live on Render. Core user-facing features include a Dashboard with live standings and upcoming race info, a Race Prediction Simulator for custom scenario modeling, a Race Data Center for historical analysis, and a Newsroom with live F1 news. An auto-updater pulls post-race results from the Jolpica-F1 API after each race weekend to keep standings and training data current throughout the season. Target ML accuracy is 45–50% for exact winner prediction and 70%+ for podium prediction, reflecting the inherent unpredictability of Formula 1.

3) Accomplishments and overall project status during this increment

Increment 3 shifted focus from integration to completion, with the primary objective of finishing all remaining features, overhauling the UI, and deploying the application to production. All four areas of the system saw significant progress, and the project crossed the finish line as a fully deployed, end-to-end functional application for the first time.

The work was distributed as follows:

Backend Infrastructure (Liv)

The backend gained several new endpoints and configuration updates to support frontend integration:

- *CORS configured for the frontend origin at localhost:5173, enabling cross-origin requests from the React dev server.*
- *GET /api/races/next is live and backed by the Jolpica API with one-hour in-memory caching, verified working.*
- *GET /api/circuits is implemented and database-backed via SQLAlchemy, returning circuit records from the local PostgreSQL database.*
- *GET /api/standings/drivers and GET /api/standings/constructors are stubbed out and routable but currently return hardcoded zeros rather than live data.*

- *POST /api/predictions/ exists and accepts the correct request payload, but returns three hardcoded driver names rather than invoking the serialized ML model.*
- *GET /api/races/upcoming was referenced in the frontend design spec but is not yet implemented in the codebase; this is deferred to Increment 3.*

ML Pipeline (Julissa)

The ML engineer retrained the model with real weather data and committed version 3 of the prediction artifact:

- *f1_winner_model_v3.pkl retrained with four real weather features (temperature, humidity, wind speed, rainfall), expanding the feature set from 10 to 12. Training workflow documented in train_model_v3.ipynb.*
- *predictor.py updated to fetch live weather from the OpenWeather API at prediction time; model_loader.py updated to load the v3.0 artifact.*
- *Simulator updated to generate weather features and pass hardcoded 2026 drivers directly, ensuring feature parity with the v3.0 model.*
- *GET /api/drivers endpoint added, returning 22 current-season drivers with team colors and flags for the simulator starting grid UI.*
- *Data contract for race_results table agreed and documented: race_id, circuit_id, driver_id, team_id, grid_position, finish_position, weather_condition, fastest_lap, points_scored, race_date — added as a comment block in predictor.py and a # DATA CONTRACT section in the backend README, signed off before the auto-updater PR was merged.*
- *Simulator endpoint validated against the redesigned frontend workflow, confirming circuit, weather, and grid parameters correctly wire through to simulate_race().*
- *Real-time race clock implemented in the Header: timezone column added to circuits table with IANA identifiers, /api/races/next updated to return circuit timezone, MY TIME and TRACK TIME displayed using Intl.DateTimeFormat for DST-aware calculations, with clearInterval cleanup on unmount.*
- *weather_service.py refactored to remove Jolpica as a weather dependency: historical functions route to Visual Crossing, forecast functions route to OpenWeather, and the response normalizer outputs an identical internal schema (temp_c, precipitation_mm, wind_kph, condition) so the ML pipeline and Simulator required no changes.*
- *Rich circuit data added to the Data Center and UI updated accordingly.*

Auto-Updater and Backend Deployment (Yulissa)

The backend was successfully deployed to production on Render:

- *Production deployment completed on Render (free tier) with PostgreSQL managed database provisioned and connected.*
- *Production database seeded with complete historical data spanning circuits, races, drivers, teams, race results, and standing tables.*
- *Docker configuration added with Dockerfile, docker-compose.yml, and .dockerignore for containerization and local development support.*

- *Database constraints added: unique constraints for driver_standings (year, driver_id), team_standings (year, team_id), and race_results (race_id, driver_id) to support upsert operations.*
- *API documentation is live at /docs endpoint with Swagger interface for all production endpoints.*
- *Weather service integration validated with both Visual Crossing(historical) and Open Weather(live) APIs operational in production.*
- *Environment variables configured with proper key management.*
- *APScheduler integrated with FastAPI lifespan management for background job scheduling.*
- *Race-based scheduling queries races.end_datetime and automatically schedules update jobs 2 hours after each race ends.*
- *Manual sync endpoint added POST /api/admin/sync?year=YYYY&round_number=N for testing and manual triggers.*

Frontend (Aleksandar & Brooklyn)

- *The frontend received a total UI overhaul that was deferred from increment-2 to avoid rushing before a grade deadline.*
- *VISUAL_STYLE_GUIDE.md was created and followed as guide to overhaul design*
- *Circuit svg images were added for use in the simulator and data center pages*
- *The data center page received full re-design and received additional circuit records to display along with circuit images.*
- *The newsroom page received a full re-design with the addition of new filters to search for news that relates to every completed race of the season.*
- *The dashboard page received a full re-design and functionality overhaul now displaying multiple upcoming races and current driver and team standings.*
- *The simulator page received a full re-design and functionality overhaul to fully connect with backend given chosen parameters and display prediction with visual ranking and details page.*
- *The frontend file structure was overhauled and routing structure was changed along with creation of layout file to maintain consistent visual design throughout different pages.*

Project Management & Architecture (Alexander)

- *The application is fully deployed and live in production for the first time — backend at racetrack-backend-2ak8.onrender.com, frontend at f1-capstone-project-ujwc.onrender.com.*
- *The team had a clear, conflict-free design spec to build from for the entire increment, eliminating ambiguity that had caused rework in earlier sprints.*
- *The race calendar table was delivered on time, unblocking Liv and Yulissa's auto-updater work downstream.*

- *The issue tracker accurately reflected team progress throughout the increment with no duplicate noise.*
- *All four pages load correctly in production with live backend data.*

Overall Status: The application is fully deployed to production for the first time, with the backend and frontend both live on Render. All four pages load with real data, the ML model is wired into the simulator, the auto-updater is scheduling post-race data pulls, and the UI overhaul has been completed across all pages. The primary known issues are the simulator occasionally routing to incorrect race data due to a circuit ID mismatch, and the weather service requiring an API key to be configured in the production environment.

4) Challenges, changes in the plan and scope of the project and things that went wrong during this increment

- *Frontend Redesign*
  - *Redesigning the frontend took more time than initially planned, along with requiring additional images and data to be sourced to fully complete the initial vision for the redesign.*
- *Duplicate GitHub Issues*
  - *A batch issue creation script was run multiple times without a title-existence guard, generating approximately 10 duplicate issues (#35–#44) that inflated the closed issue count. The script was updated with a deduplication check, and the duplicate issues were closed. Real meaningful closed issues for Increment 2 are #25, #26, and #32.*
- *DATABASE_URL Environment Variable (Resolved)*
  - *A misconfigured DATABASE_URL in .env was using the wrong PostgreSQL username format. The correct pattern (postgresql://postgres:PASSWORD@localhost:5432/f1_predictor) was documented for the team, and special character URL-encoding requirements were noted. Issue has been resolved and documented.*
- *GET /api/races/upcoming Not Implemented*
  - *The frontend design spec references a GET /races/upcoming endpoint for the dashboard upcoming race card. This endpoint was not implemented this increment and is deferred to Increment 3.*
- *Simulator Circuit Resolution Bug*
  - *The simulator was falling back to race_id: 1 (a ~2010 race) because /api/races returns paginated results of only 20 races and FALLBACK_CIRCUITS IDs did not reliably match DB circuit_id values, causing modern driver IDs to be rejected. Fixed by ensuring the simulator resolves circuits against 2026 races only.*

- *Deployment Platform Change*
  - *The original deployment plan specified Dokploy, but during implementation, it was discovered that it required a paid subscription. The team pivoted to Render, which offers a free tier and provides equivalent functionality with managed PostgreSQL and environment variable management. The change required no code modification but did shift the deployment documentation.*
- *Database Migration Synchronization*
  - *The production database initially failed to apply Alembic migrations properly because tables already existed from manual seeding. This required running manual SQL scripts to add missing unique constraints, updating the startup seeding sequence to run in the correct dependency order.*
- *Weather Data Table Empty on Production*

- *The auto-updater's weather integration initially failed in production because the weather_data table was empty (the backfill script had only been run locally. This required setting the production DATABASE_URL as a temporary environment variable in local PowerShell, running the backfill script pointing at production to populate completed races.*

- *Ipython Version Incompatibility (Resolved)*
  - *The initial Render deployment failed because requirements.txt specified ipython==9.12.0, which requires Python 3.12+, but Render uses Python 3.11. Fixed by updating ipython==8.26.0, the latest version compatible with Python 3.11.*

5) Team Member Contribution for this increment

a) Alexander Hsieh

i) I led sprint planning for Increment 3, drafting and creating 28 GitHub issues for the team with clear ownership and dependencies. I also cleaned up the issue tracker by closing 7 duplicate issues that were inflating our progress metrics, and flagged 8 component library issues that were missing entirely and blocking other frontend work.

ii) I executed the full repository restructure, reorganizing the folder layout to match our redesign spec and renaming components to align with the final page names. I generated a migration script and a guide so teammates could apply the changes without conflicts.

iii) I caught a conflict between two spec documents that were contradicting each other — one calling for a dark modern theme, the other for the retro pit crew aesthetic we had established in earlier increments. I resolved it by producing a single authoritative spec (`FRONTEND_REDESIGN_v4.md`) that retired the conflicting one and gave the frontend team a clear direction to build from.

iv) I owned the race calendar work, researching all 24 rounds of the 2026 F1 season and building a seed script to populate the database with race dates, locations, and sprint flags. This was a blocking dependency for the auto-updater, so I prioritized it to unblock teammates Liv and Yulissa downstream.

v) I deployed and stabilized the backend on Render, working through several crashes — missing database setup on startup, a broken upsert operation, and a module that wasn't included in the dependencies. I also handled port configuration for the hosting environment. The backend is live at `https://racetrack-backend-2ak8.onrender.com`.

vi) I deployed the frontend to Render, fixing a series of build errors that only surfaced in the production environment — type errors, a case-sensitive filename issue that Windows had hidden, and the API client not reading the correct backend URL. I also resolved a CORS issue caused by a trailing slash in the backend's allowed origins config. The frontend is live at `https://f1-capstone-project-ujwc.onrender.com`.

vii) I sourced the hero video for the Dashboard, debugged why it wasn't playing on mobile, and guided the team toward a CDN solution that works reliably across all devices.

viii) I documented two bugs I found during testing — a simulator issue where the wrong race data was being sent to the ML model, and a missing API key that was causing weather requests to fail in production — so they could be tracked and addressed.

b) Brooklyn Metzger

i) *I fully re-designed the data center page to match given vision along with adding circuit image display functionality.*
ii) *I fully re-designed the newsroom page to the provided specifications and design concept with the addition of adding filters that for each completed race would be added to filter by most relevant news. Added video compatibility by connecting F1 youtube channels to display latest videos.*
iii) *Completed simulator results functionality by adding successfully calling backend api to receive prediction based upon chosen weather and race line up, then visually show top three rankings in podium format and over prediction statistics and confidence bars below.*

c) *Julissa Su*
i) *I upgraded the ML model to v3.0 by retraining with four real weather features — temperature, humidity, wind speed, and rainfall — replacing the previous proxy wet_race flag, expanding the feature set from 10 to 12. I documented the training workflow in train_model_v3.ipynb and updated model_loader.py to load the new artifact. I updated predictor.py to fetch live weather from the OpenWeather API at prediction time and updated the simulator to generate weather features and pass hardcoded 2026 drivers, ensuring full feature parity between training and inference.*
ii) *I added a GET /api/drivers endpoint returning all 22 current-season drivers used by the simulator starting grid UI.*
iii) *I also agreed and documented the data contract for the race_results table, adding the canonical schema as a comment block in predictor.py and a # DATA CONTRACT section in the backend README, with sign-off confirmed before the auto-updater PR was merged.*
iv) *I also validated that the simulator endpoint's inputs and outputs remained compatible with Aleksandar's redesigned multi-step simulator frontend, ensuring the new circuit selection, weather conditions, and grid order UI correctly mapped to the simulate_race() function.*
v) *I implemented the real-time race clock in the Header, adding a timezone column to the circuits table with IANA identifiers, updating /api/races/next to return the circuit timezone, and building the ticking MY TIME and TRACK TIME display using Intl.DateTimeFormat for DST-aware calculations, with proper clearInterval cleanup on unmount.*
vi) *I also refactored weather_service.py to remove Jolpica as a weather dependency, routing historical calls to Visual Crossing and all live and forecast calls to OpenWeather, with a response normalizer ensuring the ML pipeline and Simulator required no changes.*
vii) *I enriched the circuit data available in the Data Center and updated the UI accordingly.*

d) *Yulissa Fu*
i) *I built the data seeding pipeline that populates our PostgreSQL database with F1 data (2010-2026). The seeding script has rate limiting (2 calls/second), file-based caching that reduces API calls by 85%, and automatic retry logic for reliability. The script is idempotent and safely handles re-runs without duplicates. The seeding process populates the circuits, races, drivers, teams, race_results, driver_standings, and team_standings tables with full relational integrity.*
ii) *I also updated the configuration to seed both historical race results and the current 2026 season, ensuring the standings endpoints have live data available.*
iii) *I deployed the backend to production on Render. I created Docker configuration files (Dockerfile, docker-compose.yml, .dockerignore), provisioned a managed PostgreSQL database, configured all environment variables (DATABASE_URL, VISUALCROSSING_API_KEY, OPENWEATHER_API_KEY), enabled auto-deploy from the increment-3 branch, and seeded the production database with complete historical data. I resolved multiple deployment issues including missing unique constraints, timezone column migration, ipython version incompatibility, and auto-seeding dependency order problems.*
iv) *I implemented the auto-updater system that automatically fetches and updates race results, driver standings, and constructor standings after each race. I integrated APScheduler with FastAPI lifespan management, implemented race-based scheduling that queries races.end_datetime and schedules jobs*

2 hours after race completion, added weather integration querying the weather_data table, created three fetch functions (fetch_driver_standings, fetch_constructor_standings, fetch_race_results), updated database writer functions to handle circuit_id, race_date, and weather_condition fields, and backfilled historical weather data for 2025 and 2026 using Visual Crossing API.

e) Aleksandar Stavreski

i) Completely overhauled and redesigned the dashboard with a dynamic video hero and a data-driven two-column layout, including race cards and interactive championship standings (driver and team) with color coded visualizations and fallback support. Also redesigned the Simulator, Data Center, and Newsroom pages to match the brand-new aesthetic.

ii) Developed a multi-step Simulator workflow with context-managed state, featuring circuit selection, dynamic weather conditions, and a 3-day weather widget. I had to get some help from my groupmates for this one since I had no Backend/API connectivity (due to having to remake my dev environment 3 times), but they helped me finish the final step of the simulator!

f) Olivia Reiter

i) I implemented the database write layer for the auto-updater pipeline in Backend/app/services/db_writer.py, delivering three upsert functions: upsert_driver_standings(), upsert_constructor_standings(), and upsert_race_results(). Each function accepts a list of normalized dicts from Yulissa's fetch functions, writes to the corresponding database table with ON CONFLICT handling, invalidates in-memory caches for standings and race endpoints after each successful write, and logs the number of rows affected per table.

ii) I wired the driver standings endpoint to real database data with a Jolpica fallback when the table is empty, and did the same for constructor standings, adding team color hex codes to the response via a hardcoded color map. I added a win_chance column to the driver_standings table defaulting to 0.0 as a placeholder pending championship probability implementation from the ML team.

iii) I also resolved a local database schema sync issue caused by a failed Alembic migration, manually adding the missing circuit_id, race_date, weather_condition, start_datetime, end_datetime, and is_sprint columns via ALTER TABLE commands. All three upsert functions were verified end-to-end via tests/test_db_writer.py against a live local database.

6) Plans for the next increment

-

7) Stakeholder Communication

Subject: RACETRACK (Former: F1 Race Winner Predictor) — Increment 3 Final Update

Dear Stakeholders,

I wanted to share a final update on the RACETRACK (Former: F1 Race Winner Predictor) as we close out our third and final development increment.

Increment 3 was focused on completion — finishing all remaining features, overhauling the UI, and deploying the application to production. I'm glad to report that we accomplished all three. RACETRACK is now fully live, with the backend and frontend both deployed and publicly accessible.

Here is what was delivered this increment:

On the backend, we completed the auto-updater pipeline, which pulls post-race results from the Jolpica-F1 API after each race weekend and keeps standings and training data current throughout the 2026 season without manual intervention. The race calendar database was fully seeded with all 24 rounds of the 2026 season, which unblocked the scheduler and several downstream features.

On the frontend, all four pages received a full UI overhaul aligned with our retro F1 pit crew design system. The Simulator is now fully connected to the ML model and backend, accepting circuit, weather, and starting grid inputs and returning a ranked prediction with confidence scores. The Dashboard displays live driver and constructor standings along with upcoming race information. The Data Center and Newsroom pages were redesigned and expanded with additional data sources and filtering options.

The ML model remains integrated with the simulator flow, and we are maintaining our honest accuracy targets of 45–50% for exact winner prediction and 70%+ for podium prediction, which reflect the inherent unpredictability of Formula 1 racing.

The application is live and ready for review ahead of the April 27 deadline. Thank you for your support throughout this project.

Best regards,
Alex Hsieh
Project Manager, RACETRACK — F1 Race Winner Predictor Team

8) Link to video
https://youtu.be/R_OnpfOo1z0